Blood-Brain Barrier Penetration with a Random Forest

A step-by-step report: data, model, interpretation, trustworthiness and error analysis

Headline result	Value
Test ROC-AUC (scaffold split, 408 unseen chemotypes)	0.882 (95% CI 0.847–0.914)
Balanced accuracy at tuned threshold	0.789
Best simple baseline (TPSA alone)	0.826 ROC-AUC
Known drugs held out from training, correctly classified	9 / 12

Dataset: MoleculeNet BBBP (2,050 compounds)
Model: scikit-learn RandomForestClassifier on RDKit features
Generated 04 November 2026 by `src/report.py`

Contents

Executive summary 3

1. The problem and the data 4

2. Turning molecules into numbers 4

3. Training the random forest 4

4. Honest evaluation: the scaffold split 6

5. What the model learned (SHAP) 7

6. How far can we trust it? 10

6.1 It beats the rules chemists already use . . . 10

6.2 The split matters more than the model . . . 10

6.3 Choose the decision threshold, don't assume 0.5 . . . 12

6.4 Are the probabilities trustworthy? . . . 12

7. Where the model works and where it fails 13

7.1 Applicability domain: novelty costs accuracy . . . 13

7.2 The most confident mistakes . . . 14

7.3 Label noise in BBBP . . . 14

7.4 Case studies: known drugs the model never saw . . . 16

8. Limitations 17

9. Reproducing this report 17

Executive summary

A drug for a brain disease only works if it can reach the brain, and most molecules are kept out by the blood-brain barrier (BBB). This project predicts, from a molecule's structure alone (its SMILES string), whether it crosses the BBB. It uses one random forest trained on 2,265 RDKit descriptors and fingerprint bits. The report walks through every step and then asks how far the result can be trusted.

- **Performance.** On a scaffold split, where the test molecules have core structures never seen in training, the model reaches ROC-AUC **0.882** (95% CI 0.85–0.91) and PR-AUC 0.897.
- **Better than chemists' rules.** It beats the CNS rule of thumb (0.75), TPSA alone (0.83) and a 6-descriptor logistic regression (0.81) in 100% of paired bootstrap resamples.
- **It rediscovers medicinal chemistry.** SHAP analysis shows TPSA is the top driver, with the effect flipping sign near 80–90 Å² (the textbook CNS limit is 90 Å²). H-bond donors and acids push towards "blocked"; lipophilicity pushes towards "crosses".
- **Use a tuned threshold.** The training set is 77% BBB+, so at the default 0.5 cutoff the model labels almost everything penetrant. A threshold chosen on training cross-validation raises balanced accuracy from 0.70 to 0.79 and MCC from 0.48 to 0.58.
- **Know the limits.** Accuracy drops on unfamiliar chemistry, probabilities are over-confident on new scaffolds, and the model cannot see active transport (P-gp efflux, LAT1 uptake), which explains its clearest mistakes.

1. The problem and the data

Why it matters. The blood-brain barrier is a tight layer of cells that protects the brain and keeps most molecules out. A candidate drug for a neurological disease is useless if it cannot get past it, so predicting BBB penetration early helps prioritise which molecules to synthesise and test.

Dataset. We use the BBBP set from MoleculeNet (Martins et al., *J. Chem. Inf. Model.* 2012), which labels each compound as BBB-penetrating (1) or not (0).

Property	Value
Raw compounds	2,050
Unparseable SMILES dropped	11
Molecules used	2,039
Penetrate the BBB (positive class)	76.5%
Distinct Bemis-Murcko scaffolds	1,021

Cleaning. Each SMILES is parsed with RDKit; salts and counter-ions are removed by keeping only the largest fragment. Molecules that fail to parse are dropped.

2. Turning molecules into numbers

A random forest needs a fixed-length numeric vector per molecule. Each molecule becomes **2,265 features**, built by `src/features.py`:

- **217 physico-chemical descriptors** from RDKit — interpretable quantities such as molecular weight, TPSA (topological polar surface area), MolLogP (lipophilicity), counts of hydrogen-bond donors and acceptors, and the qed drug-likeness score.
- **A 2,048-bit Morgan fingerprint** (radius 2, roughly ECFP4) — each bit marks the presence of a particular circular substructure, letting the model react to specific chemical groups.

A few descriptors can overflow on unusual molecules, so values are made finite and clipped to a safe range.

Scaffolds. Each molecule is also reduced to its Bemis-Murcko scaffold (its ring-and-linker core, stripped of side chains). Scaffolds are used to make the train/test split honest — see step 4.

3. Training the random forest

A random forest is an ensemble of decision trees, each trained on a random subset of the data and features; their votes are averaged. It handles thousands of mixed features well and needs little preprocessing, which suits this problem.

Tuning. `src/train.py` runs a 20-trial randomised search over the number of trees, depth, features per split, leaf size and class weighting. Each candidate is scored with scaffold-grouped 5-fold cross-validation on the training set only, so no test molecule ever informs the choice. The winning settings:

Hyperparameter	Chosen value
n_estimators (trees)	800
max_depth	40
max_features	log2
min_samples_leaf	1
class_weight	balanced_subsample

The best cross-validation ROC-AUC was **0.890**. The shipped model is then refit on all 2,039 molecules and saved to `outputs/rf_bbbp.joblib`.

4. Honest evaluation: the scaffold split

A **random** train/test split would be misleading here: close analogues of a test molecule often sit in the training set, so the model can almost memorise the answer. Instead we use a **scaffold split** — whole scaffold groups go entirely to train or entirely to test, so every test molecule has a core structure never seen in training. This mimics the real task of judging a genuinely new chemical series.

The split puts 1,631 molecules in train and 408 in test. It also shifts the class balance (train is 77% BBB+, test is 54%), which is exactly why accuracy at a fixed 0.5 threshold looks worse than the ranking metrics.

Metric (held-out test)	Score	95% bootstrap CI
ROC-AUC	0.882	0.847 – 0.914
PR-AUC	0.897	0.857 – 0.932
F1 @ 0.5	0.791	0.752 – 0.826
Accuracy @ 0.5	0.726	0.681 – 0.767

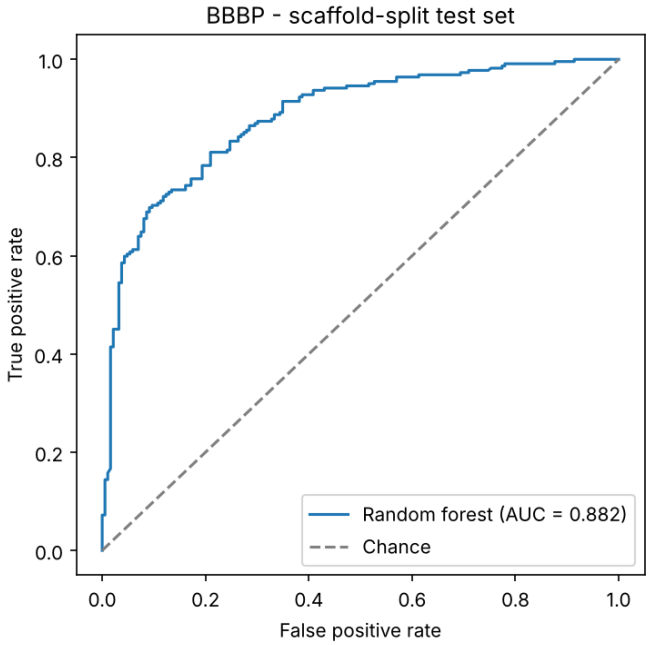


Figure 1. ROC curve on the scaffold-split test set.

5. What the model learned (SHAP)

`src/explain.py` runs exact TreeSHAP on the forest to show how each feature pushes a prediction towards "crosses" (positive) or "blocked" (negative). Reassuringly, the drivers are the ones medicinal chemists already use.

Feature	Mean |SHAP|	Direction
TPSA	0.0083	– hinders crossing
VSA_EState3	0.0073	– hinders crossing
NumHDonors	0.0062	– hinders crossing
VSA_EState5	0.0060	+ helps crossing
NumHeteroatoms	0.0060	– hinders crossing
qed	0.0059	+ helps crossing
SMR_VSA1	0.0058	– hinders crossing
NHOHCount	0.0058	– hinders crossing

- **TPSA is the #1 driver.** High polar surface area strongly lowers the predicted probability of crossing, with SHAP flipping sign near 80–90 Å² — the data rediscovering the rule of thumb that CNS drugs need TPSA below about 90 Å².
- **H-bond donors, N/O counts and heteroatoms** push towards "blocked".
- **Lipophilicity (MolLogP) and drug-likeness (qed)** push towards "crosses".

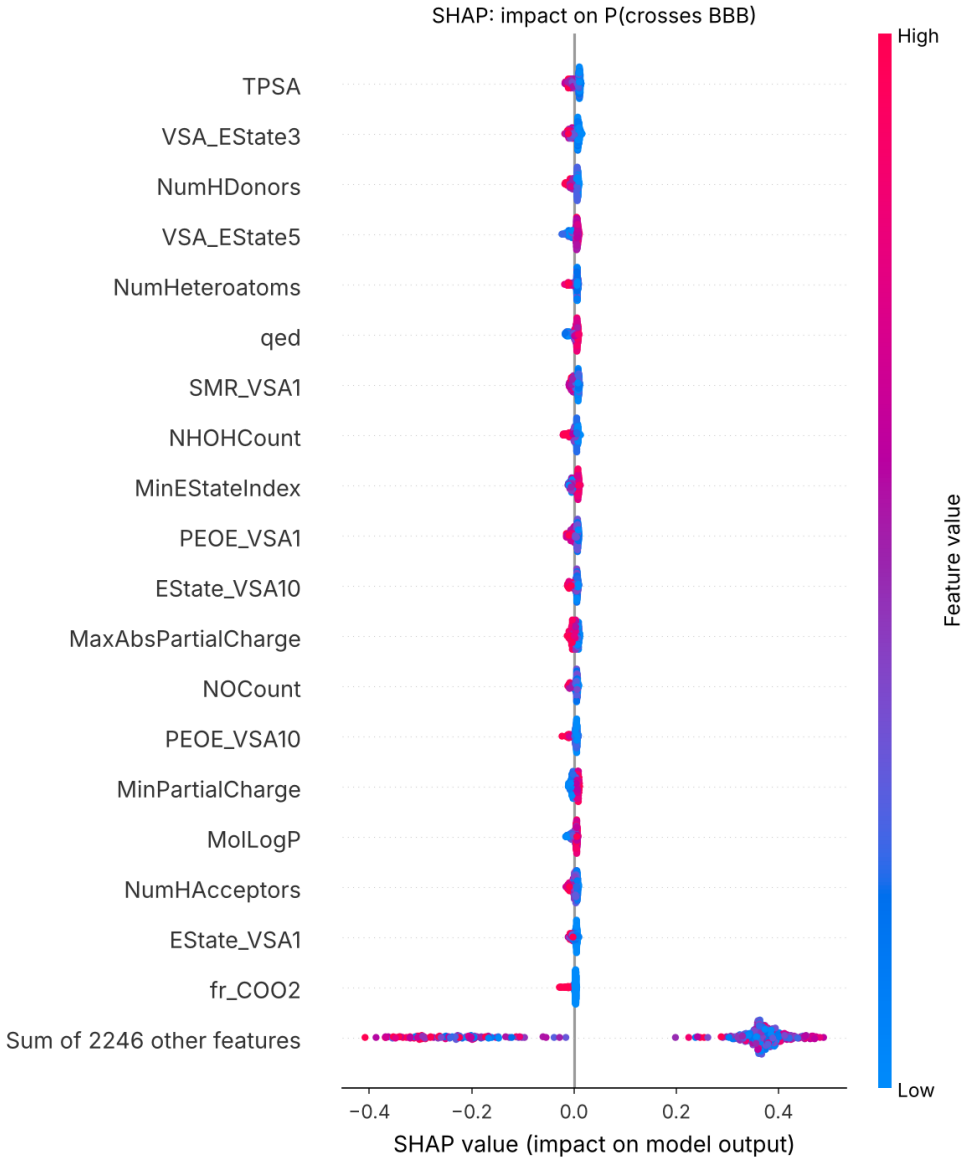


Figure 2. SHAP beeswarm: each dot is a molecule; red = high feature value.

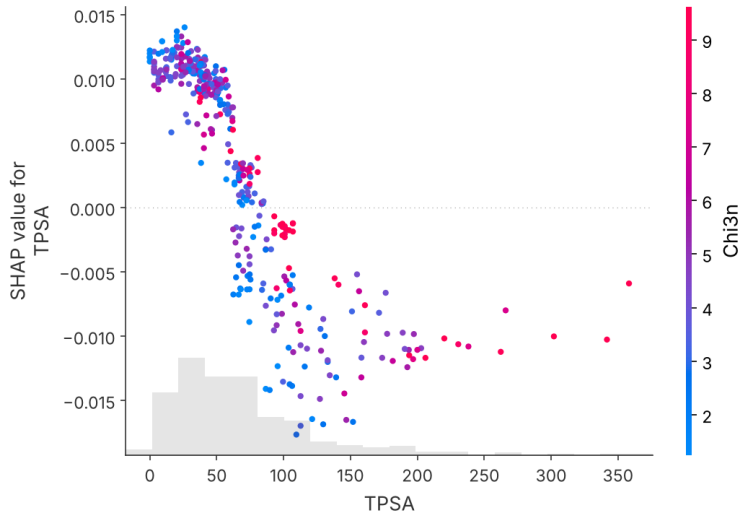


Figure 3. TPSA dependence: the effect on P(crosses) flips sign near 80–90 Å².

Explaining single molecules. TreeSHAP also decomposes individual predictions. Caffeine (P = 0.94, crosses) and penicillin G (P = 0.01, blocked) are shown below, with Morgan bits labelled by the substructure

they encode.

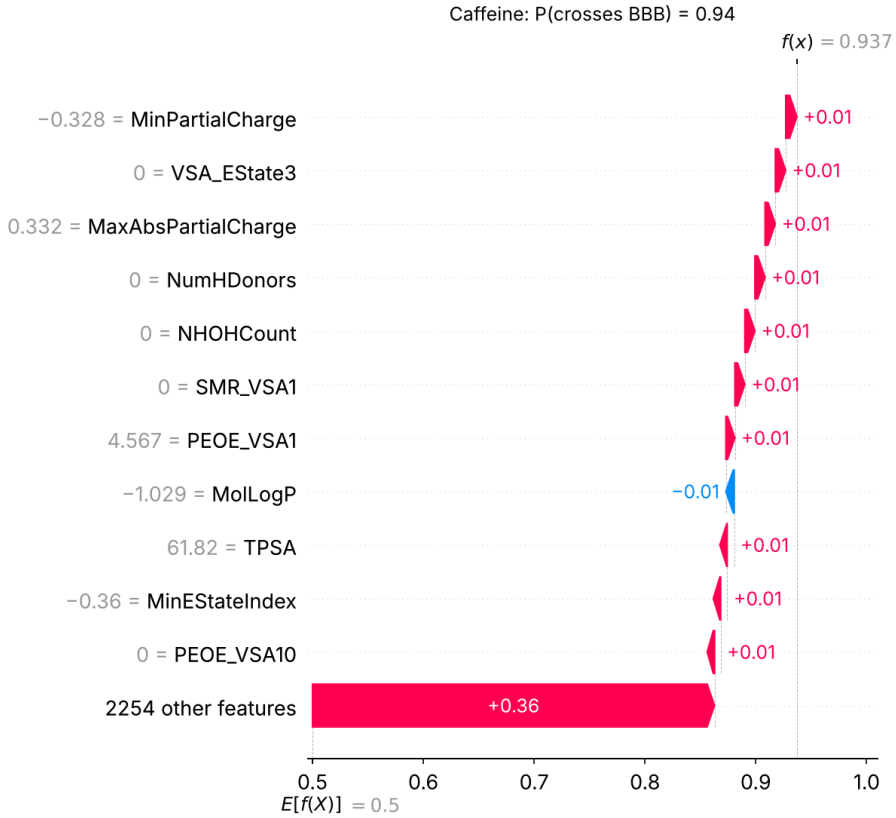


Figure 4a. Caffeine — pushed up towards "crosses".

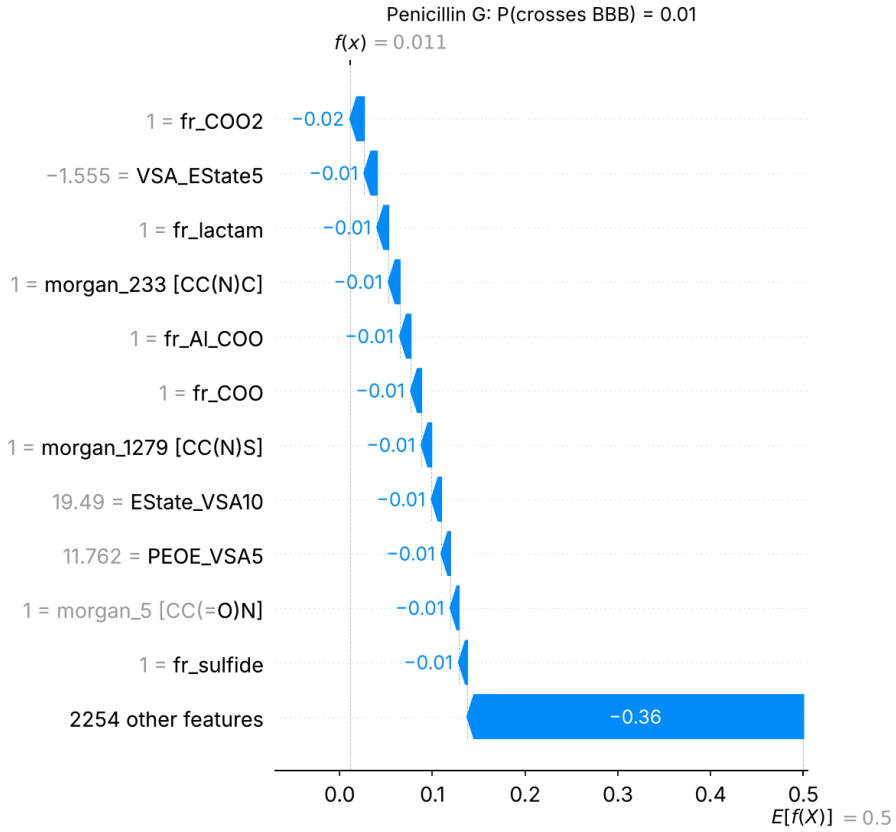


Figure 4b. Penicillin G — pushed down by its acid, β-lactam and thioether.

6. How far can we trust it?

`src/evaluate.py` stress-tests the headline number four ways.

6.1 It beats the rules chemists already use

Every model below is scored on the same scaffold-split test set. The rules use no training; the logistic regression is trained on the same molecules as the forest.

Model	ROC-AUC	95% CI	PR-AUC
CNS rule	0.747	0.70 – 0.79	0.717
TPSA alone	0.826	0.78 – 0.87	0.843
CNS MPO score	0.735	0.68 – 0.78	0.746
Logistic regression	0.810	0.77 – 0.85	0.807
Random forest	0.882	0.85 – 0.91	0.897

The forest wins against **every** baseline in 100% of 2,000 paired bootstrap resamples. TPSA alone is a strong baseline (0.83), which shows how much of BBB penetration simple polarity explains; the forest's extra ~0.06 AUC comes from substructure and descriptor interactions.

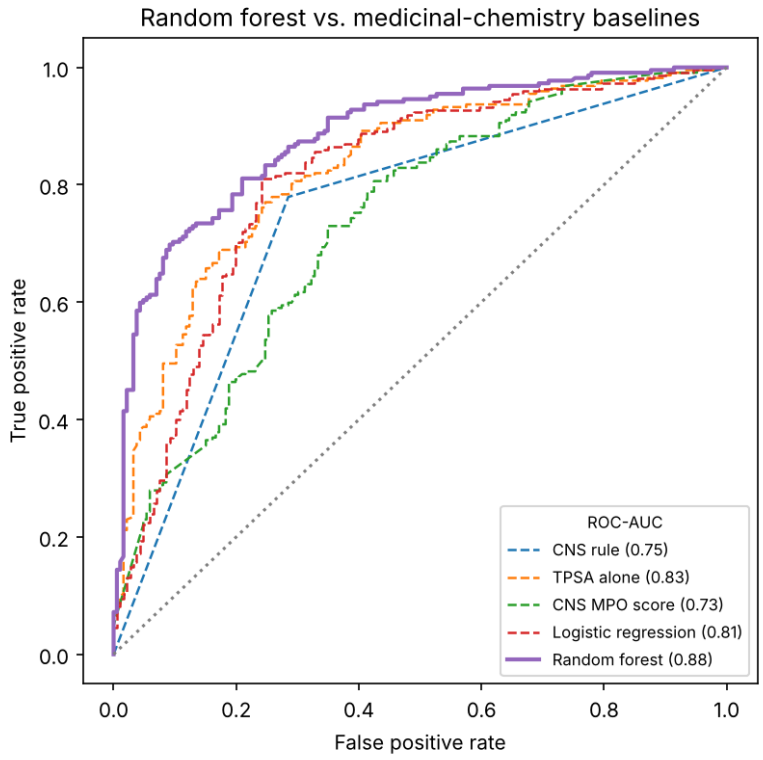


Figure 5. Random forest vs. medicinal-chemistry baselines.

6.2 The split matters more than the model

The same model, with fixed hyperparameters, refit on 5 random and 5 shuffled scaffold splits:

Split	Test ROC-AUC (mean ± sd)
Random (stratified)	0.933 ± 0.008
Scaffold, shuffled groups	0.885 ± 0.020
Scaffold, standard (reported)	0.882

A random split would overstate performance by about 0.05 AUC. The reported number sits in the middle of the scaffold-split range, so it is not a lucky split: expect roughly 0.85–0.90 on genuinely new chemotypes.

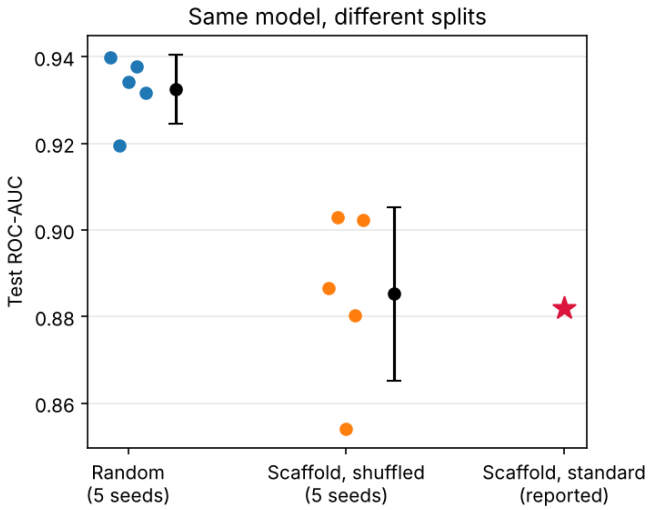


Figure 6. Same model, different splits.

6.3 Choose the decision threshold, don't assume 0.5

Because the training data is 77% BBB+, at 0.5 the forest labels almost everything penetrant (sensitivity 0.95, specificity 0.45). We picked a threshold from out-of-fold *training* predictions only (maximising balanced accuracy) and applied it unchanged to the test set.

Threshold	Accuracy	Balanced acc.	MCC	Sensitivity	Specificity
0.50 (default)	0.725	0.703	0.481	0.955	0.452
0.85 (tuned)	0.784	0.789	0.577	0.734	0.844

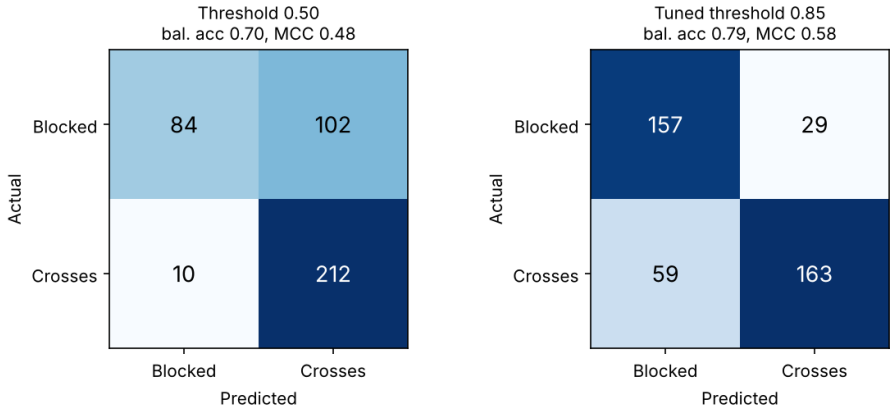


Figure 7. Confusion matrices at the default and tuned thresholds.

6.4 Are the probabilities trustworthy?

On training cross-validation the probabilities are reasonably calibrated (Brier 0.089). On the scaffold test set they are over-confident (Brier 0.182); isotonic recalibration fitted on the training CV improves this to 0.160. Read the output as a **ranking score**, not a literal probability, especially for unfamiliar scaffolds.

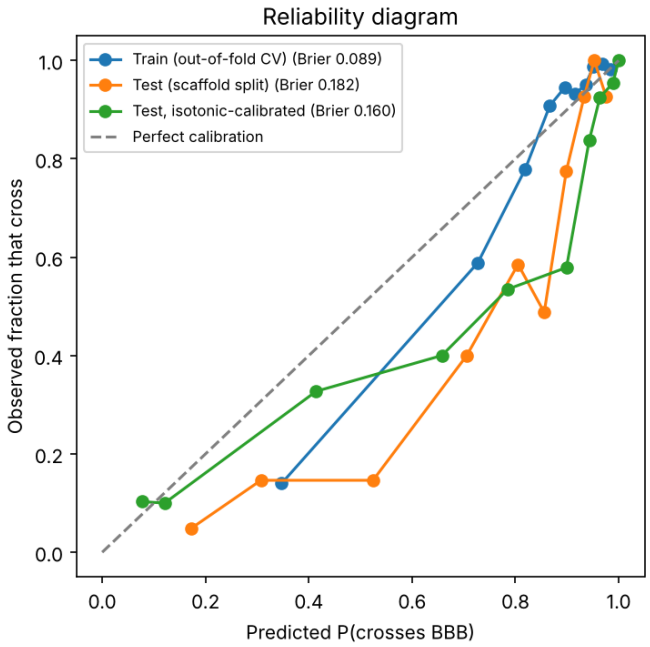


Figure 8. Reliability diagram.

7. Where the model works and where it fails

`src/analysis.py` looks at individual molecules and at the dataset itself.

7.1 Applicability domain: novelty costs accuracy

For each test molecule we take its Tanimoto similarity (Morgan fingerprints) to the nearest training molecule, then bin by that similarity.

Similarity to train	n	ROC-AUC	Accuracy (tuned)
0.0-0.3	105	0.838	0.752
0.3-0.4	86	0.792	0.698
0.4-0.5	68	0.850	0.765
0.5-0.7	118	0.918	0.847
0.7-1.0	31	0.969	0.935

Below about 0.5 similarity, accuracy falls from ~0.85–0.94 to ~0.70–0.77. A practical rule: report the nearest-neighbour similarity next to every prediction and treat anything below 0.4 as low-confidence.

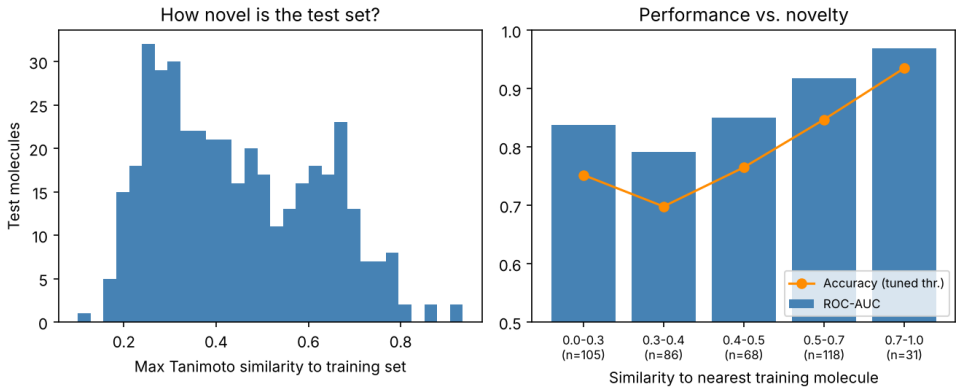


Figure 9. Test-set novelty (left) and performance vs. novelty (right).

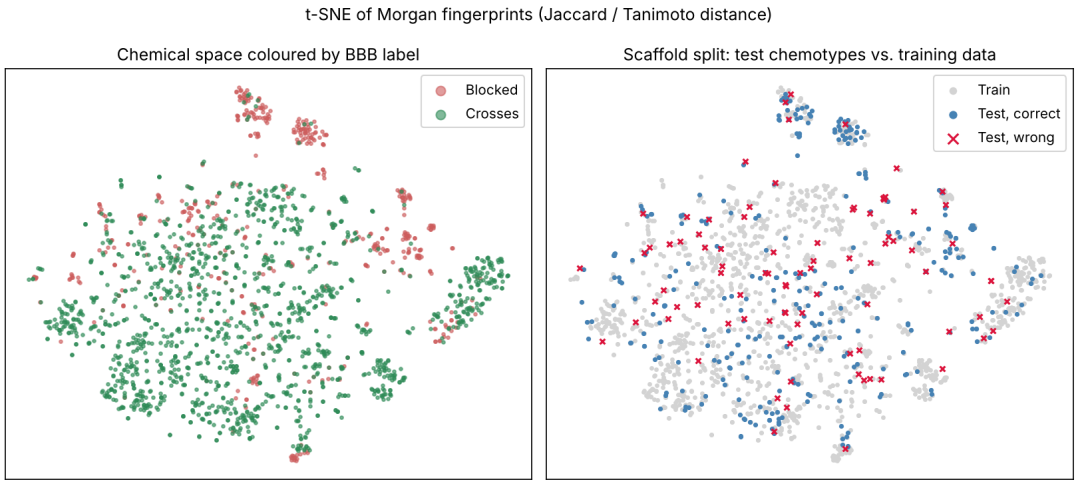


Figure 10. t-SNE of Morgan fingerprints: by label (left), by split and errors (right).

7.2 The most confident mistakes

The errors fall into two clear, chemically meaningful groups.

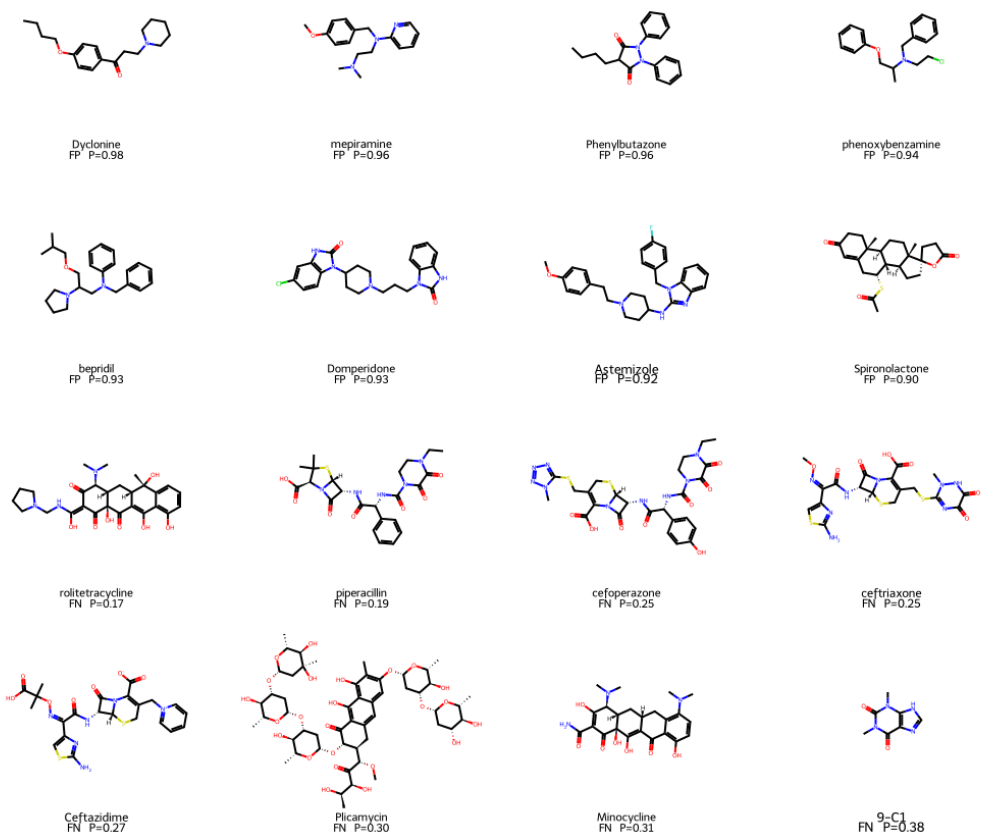


Figure 11. The most confident false positives (top) and false negatives.

- **False positives — lipophilic, low-TPSA drugs labelled "blocked"** (e.g. Dyclonine, mepiramine, Phenylbutazone). They look textbook CNS-like, but several are substrates of **P-glycoprotein**, an efflux pump that throws them back out of the brain. That is active transport, which a structure-only passive-permeability model cannot see.
- **False negatives — big, polar antibiotics labelled "crosses"** (e.g. rolitetracycline, piperacillin, cefoperazone). Their physical chemistry says "blocked", and the model agrees; the dataset label reflects clinical use against meningitis, when the inflamed barrier becomes leaky and high doses are given.

So many of the "errors" are arguably label-definition issues, not model failures.

7.3 Label noise in BBBP

There are **15 pairs** of molecules with Tanimoto ≥ 0.9 but opposite labels, and **11 of them are the exact same structure** listed twice with different labels. This noise caps the accuracy any model can reach on BBBP.

Molecule A	Label	Molecule B	Label	Same structure?
Propanolol	1	propranolol	0	yes
24959	1	Saquinavir	0	no
BRL53080	1	loperamide	0	yes
hyoscyamine	1	atropine	0	no
Quinidine	1	Quinine	0	no
SB204457	1	Colchicine	0	no

7.4 Case studies: known drugs the model never saw

Before predicting, every case-study drug was removed from the training data and the forest refit, so this is a genuine out-of-sample test against documented pharmacology.

Drug	Known	P(BBB+)	Correct	Why
Diazepam	crosses	1.00	Yes	Lipophilic sedative, passive diffusion
Fluoxetine	crosses	0.94	Yes	Antidepressant, passive diffusion
Diphenhydramine	crosses	0.98	Yes	1st-gen antihistamine (sedating)
Fexofenadine	blocked	0.47	Yes	2nd-gen antihistamine: zwitterion + P-gp efflux
Hydroxyzine	crosses	0.98	Yes	1st-gen antihistamine (sedating)
Cetirizine	blocked	0.83	Yes	Hydroxyzine metabolite: added acid limits entry
Loperamide	blocked	0.88	No	Opioid kept out by P-gp efflux
Domperidone	blocked	0.94	No	Antiemetic kept out by P-gp efflux
Dopamine	blocked	0.48	Yes	Polar neurotransmitter
L-DOPA	crosses	0.37	No	Polar, but carried in by the LAT1 transporter
Gabapentin	crosses	0.85	Yes	Amino acid, carried in by LAT1
Atenolol	blocked	0.79	Yes	Hydrophilic beta-blocker (few CNS effects)

The forest gets **9 / 12** right at the tuned threshold and correctly separates the sedating from the non-sedating antihistamines. Its failures are exactly the drugs whose BBB behaviour depends on **transporters** rather than passive diffusion (P-gp efflux, and LAT1 uptake for L-DOPA) — the clearest limitation of a structure-only model.

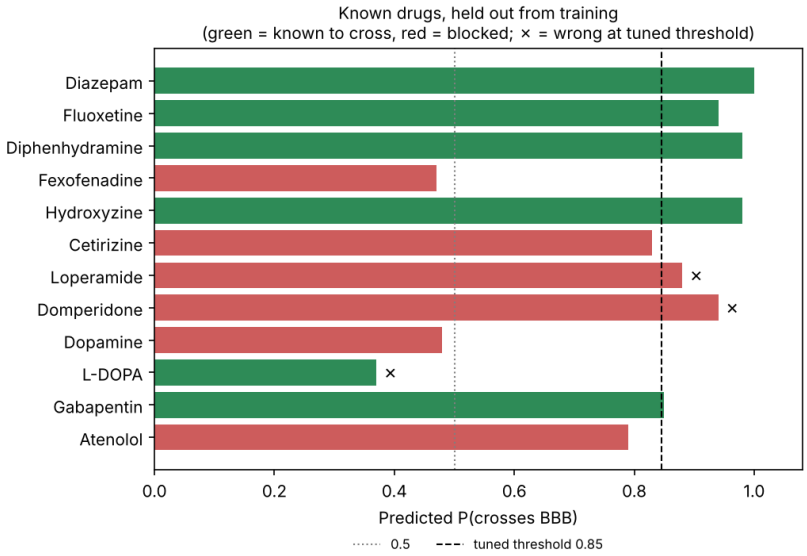


Figure 12. Known drugs held out from training (green = crosses, red = blocked).

8. Limitations

- BBBP labels are binary and come from mixed sources, with known duplicates and conflicts.
- The model learns passive permeability. Efflux (P-gp, BCRP) and uptake transporters (LAT1, GLUT1) are invisible to it unless similar molecules appear in training.
- Brain exposure also depends on dose, plasma protein binding and metabolism, none of which the model sees.
- Use it to rank and triage candidates, alongside the applicability-domain check, not as a definitive yes/no.

9. Reproducing this report

From the repository root:

```
pip install -r requirements.txt
python src/train.py    # model + metrics.json, ROC, importances
python src/explain.py  # SHAP plots
python src/evaluate.py # baselines, CIs, splits, threshold, calibration
python src/analysis.py # applicability domain, errors, label noise, cases
python src/report.py   # this PDF
```

All numbers and figures in this report are read directly from the files in `outputs/`, so regenerating those files updates the report. Generated 04 November 2026.